WELCOME TO NOWTECH

Team handbook

A practical guide to running, changing, validating, and deploying the NowTech Admin dashboard.

START HERE

1. Copy .env.example to .env
2. Add the required local values
3. Run bun install
4. Run bun run dev:local
5. Open http://localhost:3000

At a glance

Framework
Nuxt 4, Vue 3.5, Nitro SSR

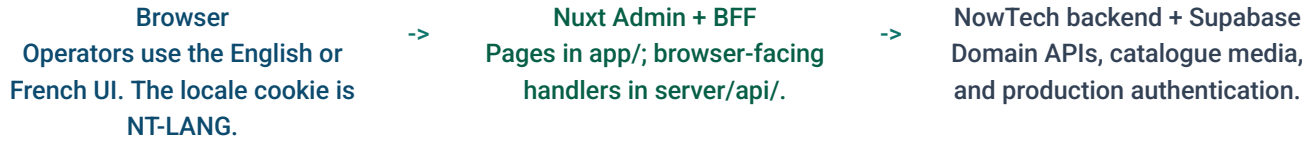
Package manager
Bun, with bun.lock as the only lockfile

Handoff format
Extracted project archive

This is a working guide, not a replacement for AGENTS.md. Read that file before changing code. Updated 6 October 2026 for the stock-status controls and the backend repository.

1. Know the system

NowTech Admin is the internal operations dashboard for NowTech. It is a Nuxt 4 application that server-renders pages and exposes a Nitro backend-for-frontend (BFF). The browser talks to the admin; the admin owns the authenticated calls to the NowTech backend and Supabase.



Project map

Location	What belongs there
app/	Nuxt pages, components, composables, plugins, schemas, stores, and browser UI.
server/	Nitro-only authentication, BFF proxy routes, startup checks, and integrations.
shared/	Types and helpers shared by app and server.
i18n/locales/	Modular English and French translation domains.
test/unit and test/nuxt	Flat unit and Nuxt component specs. The include globs are not recursive.

Security boundary

Never move backend tokens into browser code. Supabase access is created per request in server/utils/supabase.ts, and the sealed session accepts only its cookie. The BFF deliberately allowlists forwarded headers.

2. Install and configure

Prerequisites

- Bun. CI uses Bun 1.4.0; use a compatible local version.
- The extracted project archive and the configuration values supplied by the handoff team.
- A reachable NowTech backend for useful local data and sign-in.
- A local .env file. Keep it private and outside shared archives.

First run

```
cd <extracted-project-folder>
cp .env.example .env
bun install
bun run dev:local
```

bun install runs Nuxt prepare through postinstall, which creates .nuxt/. Do not use npm, pnpm, or yarn here, even though the older root README lists them.

Required local configuration

Variable	Purpose and local guidance
NUXT_SESSION_PASSWORD	At least 32 characters. Generate with openssl rand -hex 32. The server will not start with a shorter value.
NUXT_SUPABASE_URL NUXT_SUPABASE_ANON_KEY	Supabase endpoint and public anonymous key used by the production-auth path. Use team-provided values.
NUXT_API_BASE_URL	Backend origin, without /v1. For example, a local backend may be http://localhost:3001.
BACKEND_URL	Convenience source used by .env.example; NUXT_API_BASE_URL references it there.

Secret rule: .env and .env.* are ignored except for .env.example and the fake-host .env.test. Never commit a copied or real credential file.

3. Run against a local backend

The admin does not bundle the business backend; it lives in its own repository, described at the end of this section. This handbook assumes you already have a running API and valid local operator credentials. Installing the backend, PostgreSQL, and fixtures is covered by the backend's own guide.

Point the BFF at the API

```
BACKEND_URL=http://localhost:3001
NUXT_API_BASE_URL=${BACKEND_URL}
NUXT_LOCAL_AUTH_BYPASS=true
```

Then start the dashboard with bun run dev:local and visit http://localhost:3000. Use the local operator account at the login page.

What local authentication bypass does

1. In non-production only, the admin sends the login email and motDePasse to POST /v1/auth/connexion on NUXT_API_BASE_URL.
2. Token refreshes go to POST /v1/auth/rafraichir.
3. The application stores a sealed session cookie, not a browser-exposed backend token.
4. When NODE_ENV is production, the bypass is unavailable and Supabase authentication is used.

> **Run dev:local for the local backend**
>
> bun run dev:local starts the dashboard on port 3000 and clears any BACKEND_URL or NUXT_API_BASE_URL exported in your shell, so the values in .env, which point at your local backend, are the ones used. Plain bun run dev lets a variable left in your shell override .env, and the app may call a different backend.

Useful optional values

Variable	When to set it
NUXT_PUBLIC_STOREFRONT_URL	Public storefront origin used by product links. It is browser-visible.
NUXT_PUBLIC_CATALOGUE_IMAGE_ORIGIN	Origin hosting catalogue product images.
NUXT_PUBLIC_LOGO_DEV_PUBLISHABLE_KEY	Optional logo previews. Only a publishable pk_ key belongs here, never a Logo.dev sk_ secret.
NUXT_OPENAI_API_KEY	Server-only AI product-copy key. Optional for local work that does not use generation; required in production.
NUXT_PUBLIC_SUPPORT_EMAIL	Optional browser-visible support address for the login page.

The backend repository

The backend is the NowTech-backend- repository. It is edited alongside this app and is no longer treated as read-only. Its guide, documents/travailler-et-deployer.md (written in French), covers local setup, tests, and deployment. These rules matter when an admin change needs a backend change:

Rule	Detail
Branch and pull request	Never push directly to main. Work on a branch and open a pull request that says what changes and why.
A merge deploys	Merging into main is the whole release: GitHub notifies Coolify, which rebuilds the image and swaps the container, so the change is live in about two minutes. On every pull request GitHub runs typing, the migrations on a fresh database, the sql/ files applied twice, and the full suite. Do not merge a red one.
Migrations go forward only	Prisma migrations run on a database that already holds orders and money. Write them to pass on production as it is (add column if not exists): a failing migration stops the container from starting. Files in sql/ must be safe to re-run.
Local helpers	In the backend repository, pnpm dev:jeton signs a team token for the administration routes without Supabase. With SUPABASE_URL empty, the backend's own pnpm dev:local turns MOT_DE_PASSE_DEV into every address's password, and a member of boutique.membres_equipe gets their roles.

4. Daily development loop

Run the smallest useful check first. Before sharing or deploying a change, run the relevant local checks.

Command	Use it for
bun run dev:local	Start the development server at http://localhost:3000, against the backend set in .env.
bun run dev	Plain nuxt dev, without the shell clean-up: a BACKEND_URL or NUXT_API_BASE_URL already exported in your shell overrides .env.
bun run build **bun run preview**	Build the SSR app and serve the production output locally.
bunx vitest run	Run both Vitest projects once. Use bun run test:unit or bun run test:nuxt for a project. The config pins the Nuxt Image settings, so image or Supabase variables in your shell cannot change the result.
bun run lint	Run oxlint correctness checks.
bun run format **bun run format:check**	Write or verify formatting with oxfmt. Do not hand-format around it.
bun run typecheck	Run Nuxt/vue-tsc type checking.
bun run i18n:check	Check English/French key parity and raw UI strings.
bun run fallow	Audit changed code. Always use bunx fallow or the Bun scripts, never a global fallow binary.

Tests and quality gates

Before sharing a change

Run lint, format:check, i18n:check, fallow, typecheck, tests, and build as appropriate for the change.

Your team's automation

Set up your own CI and review policy to mirror the local checks. This handoff leaves automation ownership to the new team.

TDD is the default: for a function, component, or business rule, write a meaningful failing test first, then make it pass. Do not add coverage-only, snapshot-only, or framework tests.

Known test baseline

bunx vitest run currently reports 12 failing tests in six files: the four ai-product-copy specs, overview-date-field, and products-detail-form. bun run typecheck reports one error, in ai-product-copy-service. All of them predate the stock-status work and are recorded in reports/2026-10-05-assessment-vitest-failures.md and reports/2026-10-05-assessment-typecheck-failure.md. A few heavy Nuxt specs can also time out when the whole suite runs under load; rerun the file on its own before treating it as a regression.

5. How we change code

Working agreements

Area	Working rule
Server state	Use TanStack Query. Pinia is for browser-only state such as preferences and draft filters, not a mirror of API data.
Components	Create one folder per component. Do not edit app/components/ui/ unless the user explicitly asks; it is generated shadcn-vue output.
i18n	Use NuxtLinkLocale for internal links and custom localized routes. Small, stable components may use per-component messages.
Styles	Use Tailwind utilities and existing CSS tokens. Tailwind is configured in app/assets/css/main.css, not a tailwind.config file.
Tests	Unit specs are Node; components use Nuxt + happy-dom. Keep files directly in test/unit or test/nuxt because the globs are flat.

Delivery flow

1 Plan -> 2 Test -> 3 Implement -> 4 Review

A plan, implementation, and review share one kebab-case slug. Plans go in plans/, implementation walkthroughs in walkthroughs/, and reviews in reviews/. Investigations without a plan go in reports/.

Keep each change narrow, explain its validation, and preserve other contributors' work.

6. Product stock status

Two things describe a product's stock: the quantity counted, and whether the shop sells it. Since 6 October they are separate. The Overview tab puts the delivery time first and the two controls below it, side by side; the Stock tab holds the count and its movements.

The two controls

Control	What it does
In stock / Out of stock	The selling status. Out of stock hides the product from the storefront and makes new carts refuse it. While a product tracks its quantity the switch is locked, because the count decides.
Track quantity	On means the product is counted and follows its count. Turning it off keeps today's status as an explicit decision, so customers see no change; turning it on hands the decision back to the count. A product that was never counted cannot be tracked until you count it with the pencil beside the quantity.
Status badge	Names who decides: Follows the count, From WooCommerce, or Set in the panel. The last one is shown in a warning tone because a panel decision overrides every WooCommerce import.
Not counted badge	Marks a product that has never been counted. Its quantity reads 0, muted.

Edge cases

Situation	What happens
Status contradicts the count	A warning dialog quotes the count, for example when hiding a product that has 7 in stock. Nothing is written until staff confirm.
Follow WooCommerce	A button on the Stock tab, offered only for an uncounted product whose status was set in the panel. Its dialog names the status being dropped; Confirm clears the decision, so the shop uses WooCommerce's status again, now and after every import. A counted product hands the decision back with Track quantity instead.
New product	The quantity starts empty. Leave it empty to sell without counting, and pick a Status (In stock by default); enter a quantity to count the product and let the quantity decide.
Orders already placed	An Out of stock set in the panel hides the product and refuses new carts, but staff can still confirm an order the customer placed earlier. A product that is out of stock because its count is zero, or whose count cannot cover the order, still blocks the confirmation.
Mark as restocking	Hidden for a product that was never counted, and while a panel decision is in force.

Backend contract

The decision lives in its own table, boutique.produit_disponibilites, and is written by PATCH /v1/catalogue/administration/produits/:id/disponibilite. The call is direction-only, never creates a stock row or a movement, and is journalled as stock.disponibilite_modifiee. The table sits outside what the WooCommerce ingestion replaces, so a decision survives a re-ingestion.

7. Rules that bite

These constraints are deliberate. Treat them as design boundaries, not cleanup opportunities.

Do this	Because
Keep zod at ^3.25.76	nuxt-zod-i18n depends on zod 3 runtime exports. Zod 4 causes a Nitro production import failure.
Keep TypeScript at ^5.9	vue-tsc 3 resolves a TypeScript path that TypeScript 7 no longer exports.
Keep createAuthClient() a factory	A singleton can retain a session in memory and leak one request's context to another.
Keep sessionHeader: false	It prevents an x-nt-session header from bypassing the cookie-only session boundary.
Use the BFF header allowlist	proxyRequest would forward the sealed Cookie by default. The explicit proxy headers protect that boundary.
Prefetch only above-the-fold data	The shared backend throttler is 10 requests per second. A six-query SSR dashboard is a design error.
Preserve backend error detail	French backend details remain beneath translated status headlines because the backend does not yet supply stable error codes.
Set status through the decision, not the count	Never write a movement or a stock row just to flip a status. Use the disponibilite endpoint, so the register keeps recording counts only.
Write translation keys as literal t() calls	scripts/check-translation-keys.mjs, run by i18n:check, cannot verify a key built from a template string.

API safety

nuxt-security disables its XSS validator only below /api/** because it can rewrite valid JSON values. Backend validation remains the input boundary. Do not broaden that exception.

If an instruction seems unexpected, find its rationale in AGENTS.md or the linked postmortems before changing it.

8. Build and deploy

This is an SSR Nitro application. Do not treat nuxt generate as the normal production deployment path.

Production build

```
bun install --frozen-lockfile
bun run build
node .output/server/index.mjs
```

For a local smoke check after building, run bun run preview. Nitro honours PORT or NITRO_PORT and HOST or NITRO_HOST when serving the output.

Production configuration checklist

- Set the required session, Supabase, API-origin, catalogue-image, and storefront values from the environment section.
- Set NUXT_OPENAI_API_KEY when deploying. The startup configuration check requires it in production, even when local development can omit it.
- Keep every secret server-only. Public NUXT_PUBLIC_* values are intentionally exposed to the browser.
- Set NUXT_BACKEND_SHARED_SECRET when the backend is configured to recognise this server's requests outside its normal shared rate limit.
- Do not depend on NUXT_LOCAL_AUTH_BYPASS in production. The code blocks it when NODE_ENV is production.

> **Deployment boundary**
>
> The handoff materials do not establish a hosting project, deployment workflow, domain configuration, approval path, or rollback procedure. Agree these operational details with the new owning team before releasing. Do not invent them.

Before deployment

Confirm the build succeeds, the deployed environment has every required secret, the target backend origin is correct, and the deployment owner has a rollback path. Keep the AI product-copy model and its matching input/output cost values aligned before relying on audit cost figures.

The backend ships on its own: a merge into its main branch deploys it. When an admin change calls a new backend endpoint, as the stock-status controls do, the backend change must be live first.

9. Where to look next

Use the handbook to get productive, then use the source of truth for the detail.

Read this	When you need
AGENTS.md	The complete project conventions, command list, architecture, testing rules, and path-specific instructions.
.env.example	Current variable names, local defaults, and comments about their safe use.
package.json	The supported local commands for validation, formatting, testing, and building.
plans/, walkthroughs/, reviews/	Precedents for planning, implementation notes, and review depth.
reports/2026-08-27-postmortem-zod-4-production-crash.md	Why zod must stay on the supported 3.25.x bridge release.
reports/2026-09-01-postmortem-node-26-localstorage-plugin-chain.md	The Node 26 / localStorage plugin-chain failure and its production significance.
reports/2026-10-01-assessment-local-backend-merge-runtime-state.md	Known local backend schema, fixture, order-reference, and banner-runtime context.
plans/ and walkthroughs/ 2026-10-06-decouple-stock-status.md	The design, the deviations from it, and the live checks behind the stock-status controls.
NowTech-backend-/documents/travailler-et-deployer.md	How the backend ships, its migration and SQL rules, and the stock-status contract. Written in French.

A final checklist

1. Read AGENTS.md before touching code.
2. Use Bun and keep credentials in .env only.
3. Run the local backend path with NUXT_API_BASE_URL and non-production auth bypass when appropriate.
4. Write meaningful tests first for changed behaviour.
5. Run focused checks, then the relevant full gates.
6. Run the relevant local gates and record the validation outcome.
7. Ask before making assumptions about production hosting or the sibling backend.

Welcome to the team. Make the next change easier for the person after you.